

This assignment focuses on strings, file I/O, and input validation through implementing a complete text-based Wordle game. You will create a single Java program that reads word lists from CSV files, validates user input thoroughly, calculates feedback for guesses, and provides a complete game experience.

**Word Lists:** The assignment uses word lists from the Kaggle dataset (provided via Moodle!):

<https://www.kaggle.com/datasets/bcruise/wordle-valid-words>.

The game uses two CSV files containing valid words:

- `valid_solutions.csv`: Contains all valid solution words (words that can be the answer). These are valid words that can be guessed.
- `valid_guesses.csv`: Contains all valid guess words (words that players can guess). Note that solution words are **NOT** included in this file, but they are still valid guesses.

Each CSV file has a header row "word" followed by one word per line. Since solution words are also valid guesses, you should merge them into a single list for validation purposes.

**Submission Requirements:**

- Read the `Organizational Guidelines` on Moodle, and adhere to them!
- Submit a PDF document containing your **solution idea, code, and tests**
- Include all Java source files (.java) in a ZIP archive
- As tests include copies or screenshots of the console output
- Use the naming convention: `MC_UE05_LASTNAME.pdf` and `MC_UE05_LASTNAME.zip`

**Hint:** You can reuse concepts from class exercises, such as file reading patterns, search and sort, input validation strategies, and string manipulation techniques. Don't reinvent the wheel - build on what you've already learned!

---

**Important:** This is a single assignment implementing one complete Wordle game. The exercises below show how points are distributed across different aspects of the implementation. Think about:

- How to read and handle CSV files gracefully
- What validations are needed for user input
- How to calculate accurate feedback (especially with duplicate letters)
- How to structure your code for clarity and maintainability
- How to handle errors and edge cases

### **Exercise 1 (5 points) File I/O and Error Handling**

Read the CSV files into your program and make sure to handle failures gracefully. Think about what can go wrong when reading files and how to handle those situations appropriately.

### **Exercise 2 (5 points) Input Validation**

Validate guesses as much as you can. Think about what could go wrong with user input and handle it appropriately. When checking if a word is in the valid word list, think of how you can do this efficiently.

### **Exercise 3 (5 points) Feedback Calculation**

Calculate and display feedback for each guess. Make sure to handle duplicate letters correctly.

### **Exercise 4 (5 points) Autosolve Feature**

When a user types "HELP!" as their guess, automatically suggest a valid word and use it as their guess. (Just a random choice is valid and worth only 2 points, but you can do MUCH better!)

### **Exercise 5 (4 points) Complete Game Implementation**

Implement the complete game with all features working together.